

YouTube link: https://youtu.be/VFEtVSo0_g4?si=SrEAfet4FhhoQJdL



Introduction:

This project aims to balance "Rocky," a balancing robot that functions as an inverted pendulum on a two-wheeled cart. The goal is to design control systems that maintain the robot in an upright position even when external disturbances impact the system, using both analytical and numerical tools on a real physical system.

We created a first-order model to describe the cart's motor behavior.

$$M(s) = \frac{Y(s)}{V(s)} = \frac{\alpha\beta}{s + \alpha} = \frac{\frac{K}{\tau}}{s + \frac{1}{\tau}}$$

$V(s)$ is the actual wheel velocity and $Y(s)$ is the commanded velocity. We then calibrated the motor by sending alternating square-wave commands to the motor and measuring the resulting wheel speeds.

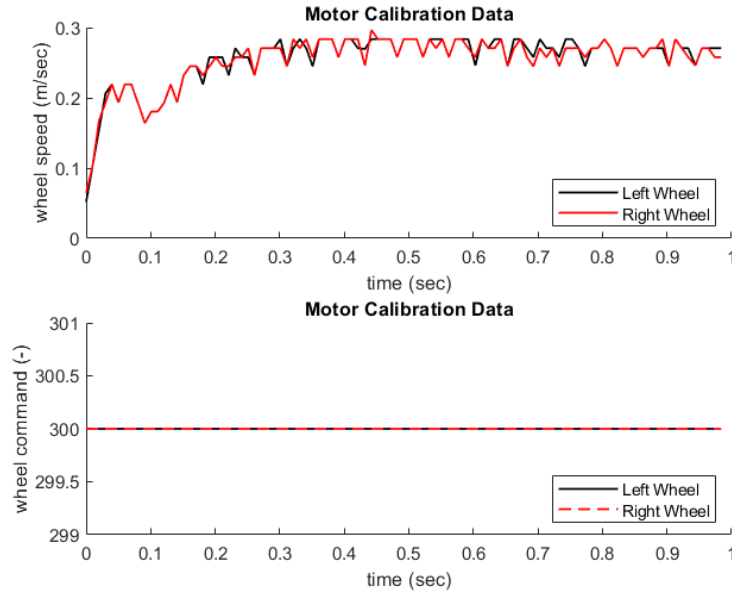


Figure 1: Motor Calibration Data

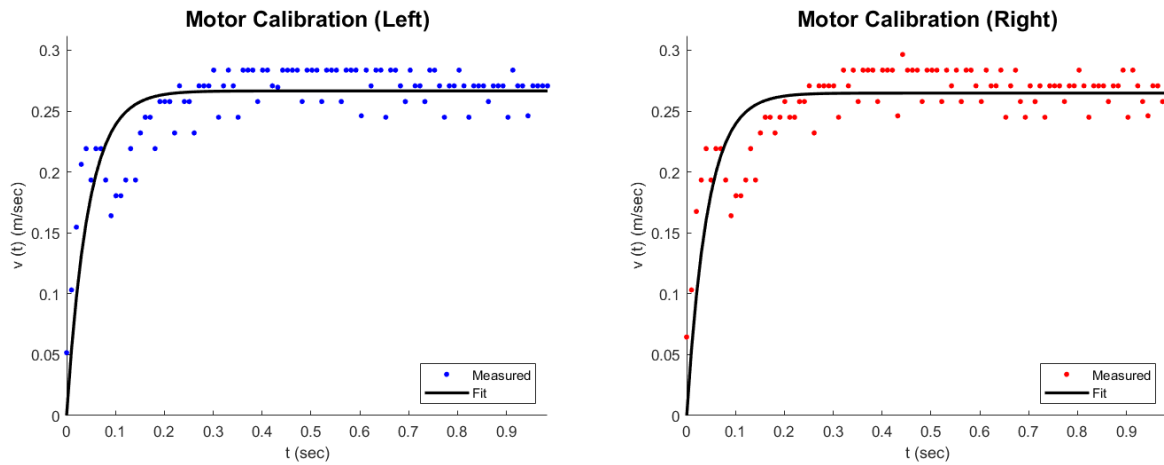


Figure 2: Motor Calibration Data (Left and Right)

We then created a line of best fit on the collected input-output measurements, as we can see in figure 2, and the time constant (τ) and gain (K) were estimated, which resulted in

$$\tau_l = 0.44264 \quad \text{and} \quad \tau_r = 0.42575$$

$$\alpha_l = 22.5917 \quad \text{and} \quad \alpha_r = 23.4881$$

$$\beta_l = 0.00088851 \text{ and } \beta_r = 0.00088261$$

The Rocky acts as a pendulum modeled by a second-order differential equation

$$\ddot{\theta} + 2\zeta\omega_n\dot{\theta} + \omega_n^2\theta = 0$$

We then found the angular frequency by holding one end of the Rocky and having it swing freely about one of its ends. The oscillations was found by using the Gyro on the Rocky, which helped find the effective length using

$$\omega_n^2 = \frac{g}{l_{eff}}$$

Natural Frequency: 4.6078 (rad/sec)

Effective length: 0.4620 (m)

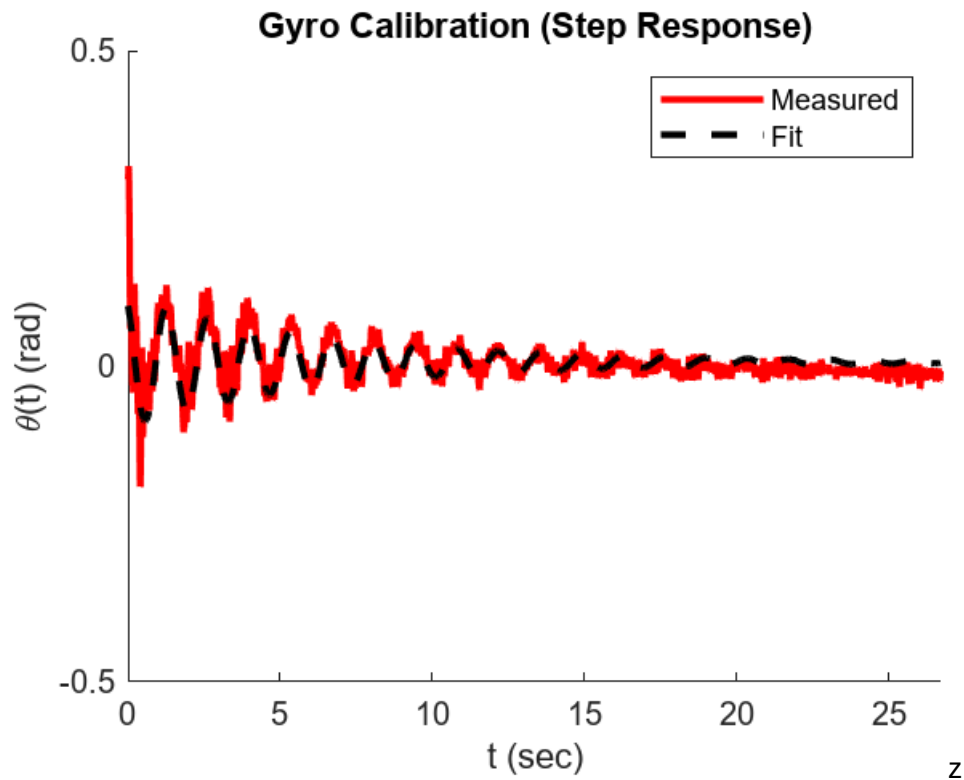


Figure 3: Gyro Data

Now we need to create a control function for the Rocky. We first derived the nonlinear equation of motion for the Rocky system, modeled as an inverted pendulum on a cart.

By analyzing the free-body diagram of the pendulum, the net moment with respect to the pivot point is determined by the gravitational force acting on the center of mass:

$$\sum \vec{\tau}_A = mgl \sin(\theta) \hat{k}$$

The angular momentum of the pendulum with respect to the point is given by:

$$\vec{H}_A = (I_A \dot{\theta} - ml * \dot{x} * \cos(\theta)) \hat{k}$$

We applied the angular-momentum principle and took its derivative, yielding a general equation of motion. After canceling terms, we arrive at the nonlinear ordinary differential equation (ODE) for the system:

$$\ddot{\theta} - \frac{g}{l_{eff}} \sin(\theta) = \frac{\ddot{x}}{l_{eff}} \cos(\theta)$$

In order to linearize the system, we looked at small angular displacements from the upright equilibrium position.

For small values we applied the small-angle approximations:

$$\sin(\delta\theta) \approx \delta\theta$$

$$\cos(\theta + \delta\theta) \approx 1 - \frac{\delta\theta^2}{2}$$

By substituting these approximations back into our motion equation and discarding the higher-order terms, we obtained the linearized equation of motion:

$$\delta\ddot{\theta} - \frac{g}{l_{eff}} \delta\theta = \frac{\dot{v}}{l_{eff}}$$

(Where $\dot{v}$ is the horizontal velocity of the cart).

To convert the linearized ODE into a transfer function, we took the Laplace transform of the equation, assuming zero initial conditions:

$$s^2\theta(s) - \frac{g}{l_{eff}}\theta(s) = \frac{s}{l_{eff}}V(s)$$

Finally, rearranging this relationship to solve for the ratio of the output angle to the input cart velocity provides the open-loop transfer function for the cartpole pendulum:

$$H_{v\theta}(s) = \frac{\theta}{V(s)} = \frac{s/l_{eff}}{s^2 - \omega^2}$$

The goal of the initial system is to maintain the body in a vertical position and react to any external disturbances. The system was stabilized by using a closed loop with a Proportional-Integral (PI) controller:

$$K(s) = K_p + \frac{K_i}{s}$$

The system was designed with a target decay rate and oscillation frequency.

The average of the three closed-loop poles is fixed at $\frac{\alpha}{3}$ the dominant pole. The poles were placed at:

- $p_1 = -7.53$
- $p_2 = -7.53 - (0.000033i)$
- $p_3 = -7.53 + (0.000033i)$

Assuming the imaginary component is zero, the constants were:

$$K_p = \frac{l_{eff}}{\alpha\beta} \left(\frac{\alpha^3}{3} + \omega^2 \right) = 4404.4$$

$$K_i = \frac{l_{eff}}{\alpha\beta} \left(\frac{\alpha^3}{27} + \alpha\omega^2 \right) = 2087.0$$

The system was then modeled in Simulink

The three-pole model uses the difference between the current angle and the ideal angle as the input to a PI controller, a motor transfer function, and a velocity to angle transfer function. Because velocity is the output of the motor transfer function, we can take the velocity and integrate to take the position from there. The theta we take from the velocity to theta transfer function. D(s) was modeled using a square wave with an amplitude of 0.1 and a period of 5 seconds. The system models a rocky that doesn't try to maintain a constant x value, so a theta value that returns to and stays at zero is expected, along with an increasing velocity as the disturbance is applied and an x value that increases with an increasing velocity. Plots are below.

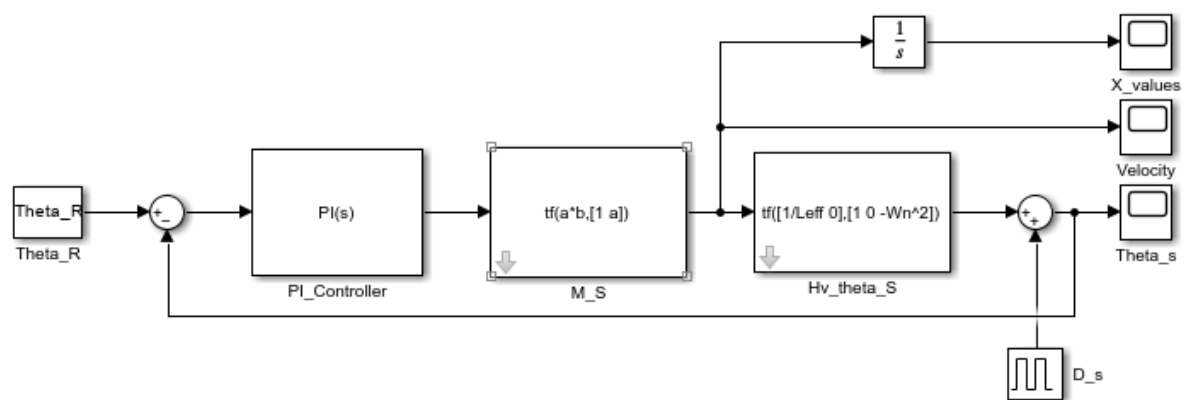


Figure 4: Simulink Block Diagram for 3 Pole

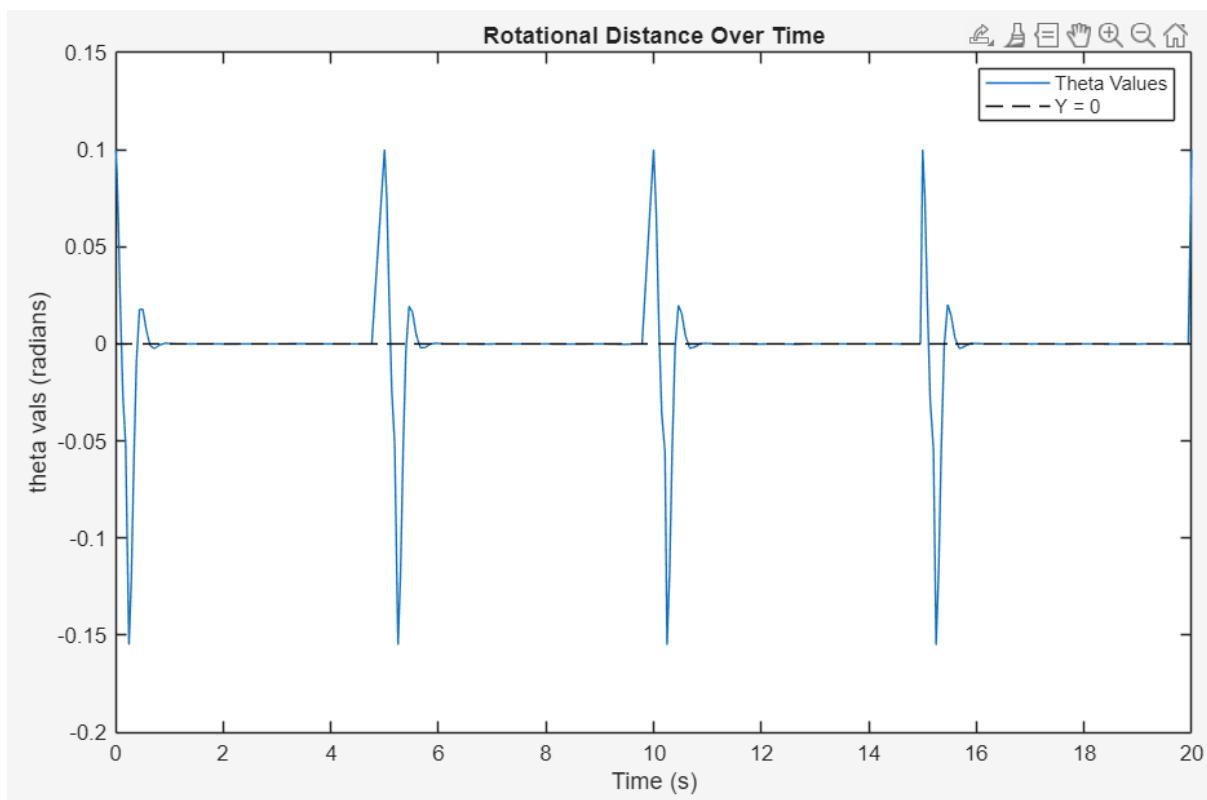


Figure 5: Theta vs Time from 3 Pole Simulink Simulation

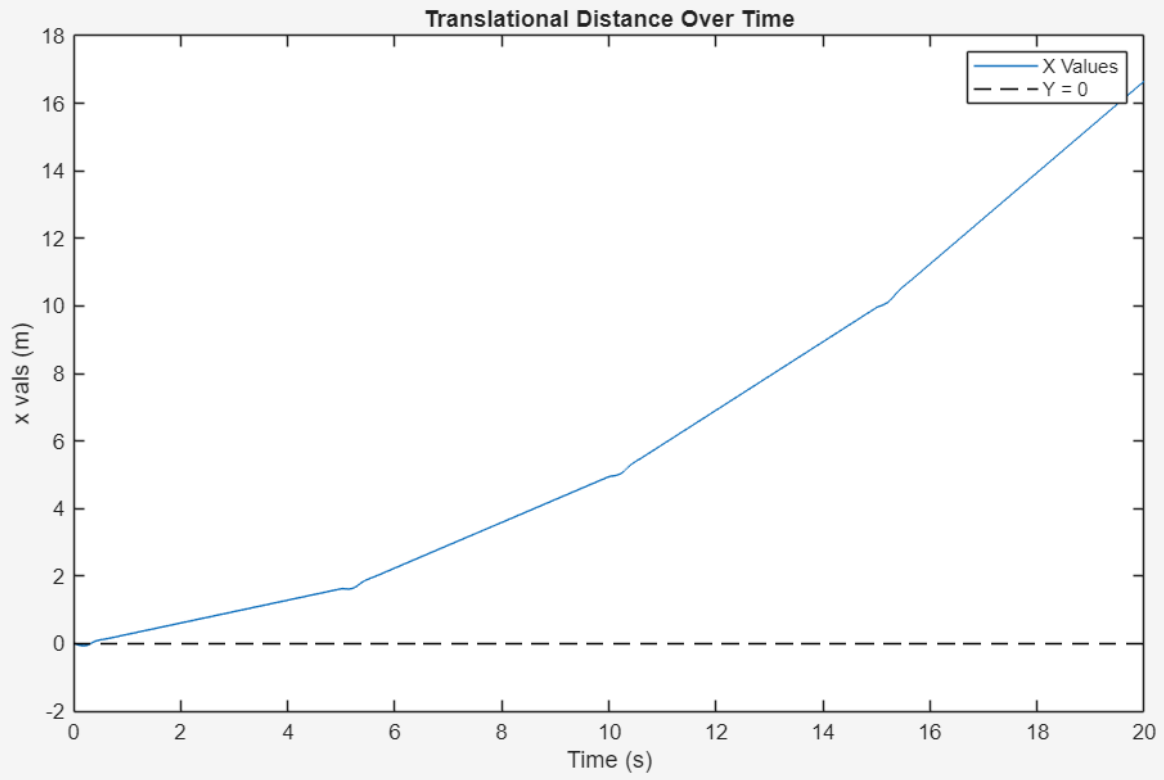


Figure 6: Distance vs Time from 3 Pole Simulink Simulation

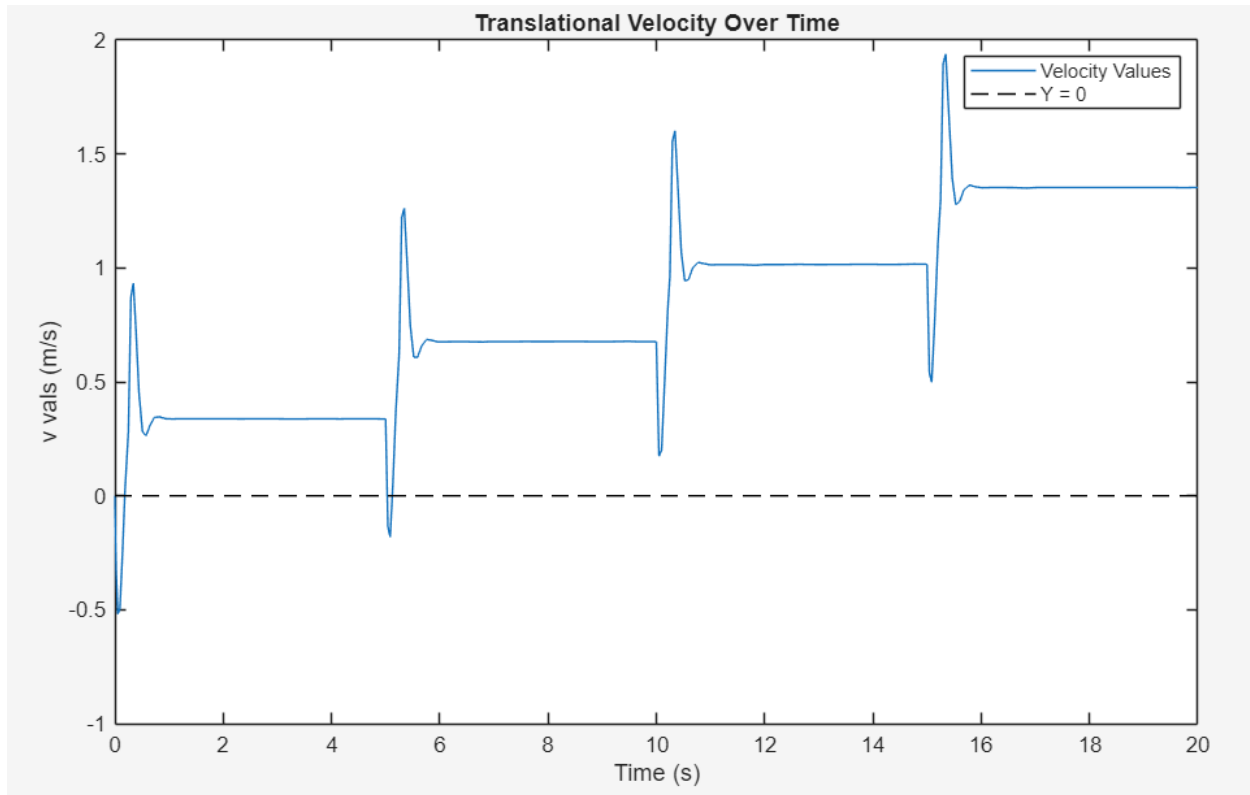


Figure 7: Velocity vs Time from 3 Pole Simulink Simulation

Then we began to work on keeping Rocky in the same place while balancing itself. We added an additional feedback loop to control the robot's position and velocity

The controller is represented by

$$J(s) = J_p + \frac{J_i}{s} + \frac{C_i}{s^2}$$

We then took this and implemented the controller in the time domain as follows:

$$y(t) = -J_p \Delta v(t) - J_i \Delta x(t) - C_i \int \Delta x(t) dt$$

We did not find the poles in a conventional way. We started with initial estimations that was given inside the 'Rocky_5_closed_loop_poles.m'. We adjusted the values by interpreting performance issues with the cart. The following

- Observation: The robot was initially too sluggish and fell over before the wheels could drive under its center mass

- Adjustment: We moved the Dominant poles more positive to increase reaction time
- Observation: The robot was excessively rocking back and forth
 - Adjustment: we moved the poles slightly more positive, slightly cause it was pulling to much power
- Observation: We did some research and found that the non-imaginary poles should be grouped together so they do not cancel each other out.
 - Adjustment: we grouped the real poles closely together, keeping their values within one unit of each other.

We found the pole locations to be

- $p_1 = -2 - 2i$
- $p_2 = -2 + 2i$
- $p_3 = -8$
- $p_4 = -7$
- $p_5 = -6$

Using the 'Rocky_5_closed_loop_poles.m' MATLAB script that uses guess and check to find them. We then gathered the control constants for the system as well, and they were:

- $K_p = 8689$
- $K_i = 40173$
- $J_p = -5893$
- $J_i = 119$
- $C_i = 6306$

The system was then modeled in Simulink, very similar to the three-pole system. The main addition to this system was the “keep it in place” PID controller. The difference between the reference velocity and current velocity is plugged into the same motor controller transfer function as the three pole system to get the desired velocity out of the motor. Using this system, the rocky stays in the same place as it remains standing, instead of using forward momentum to prevent falling. Plots below.

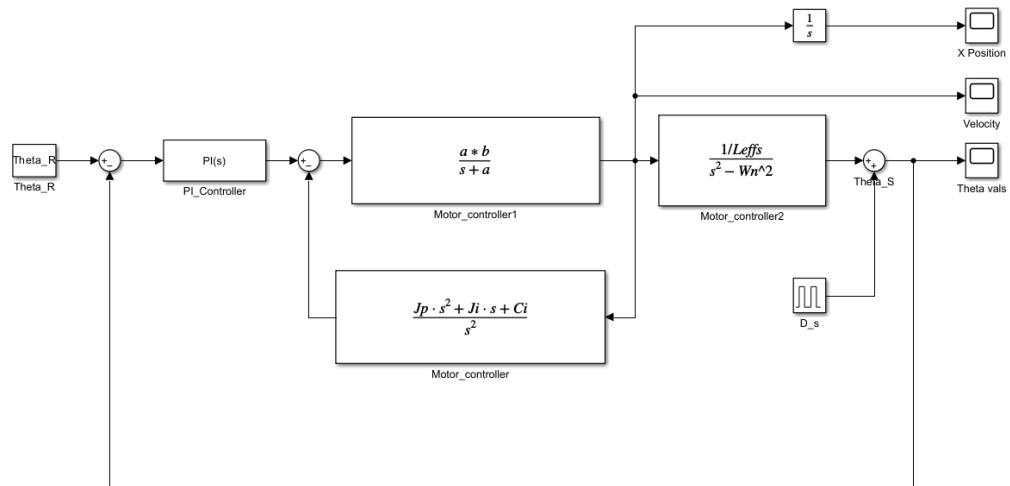


Figure 8: Simulink Block Diagram for 5 Pole

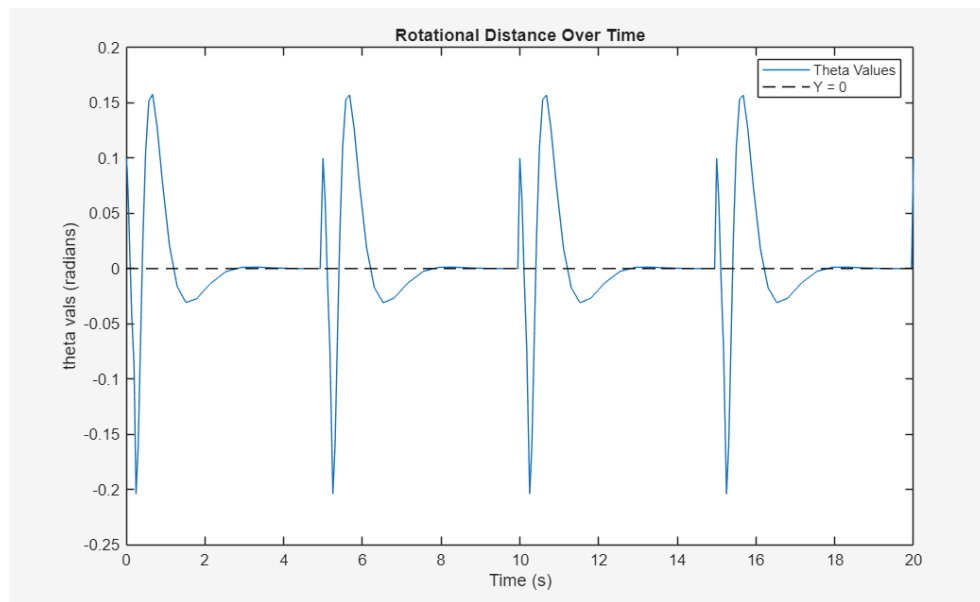


Figure 9: Theta vs Time from 5 Pole Simulink Simulation

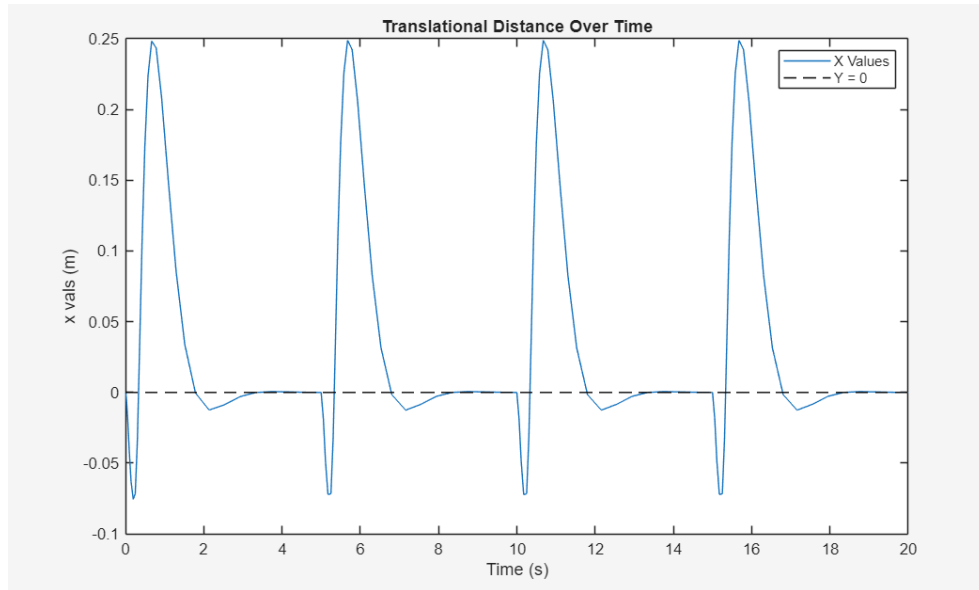


Figure 10: Distance vs Time from 5 Pole Simulink Simulation

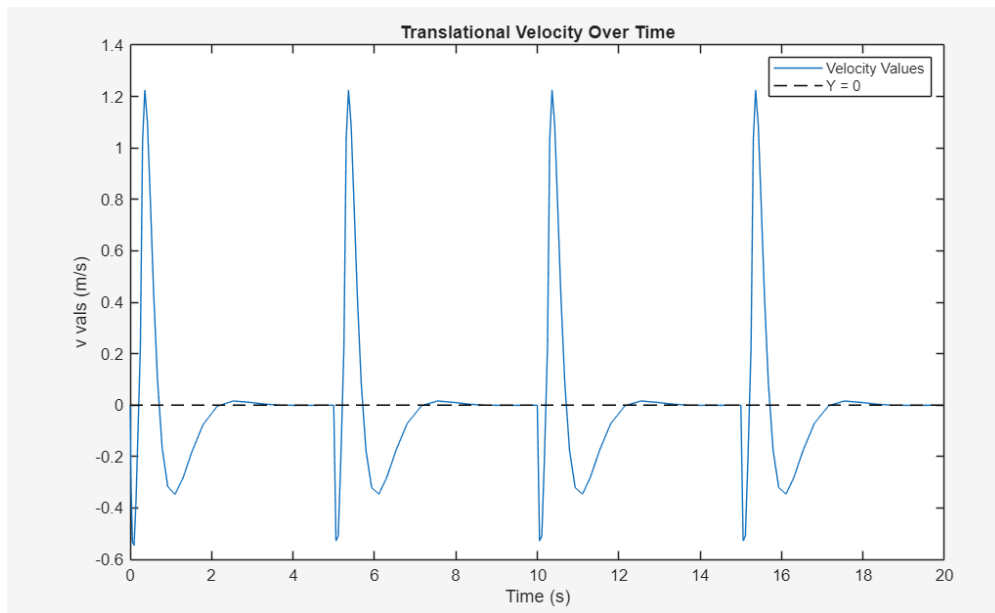
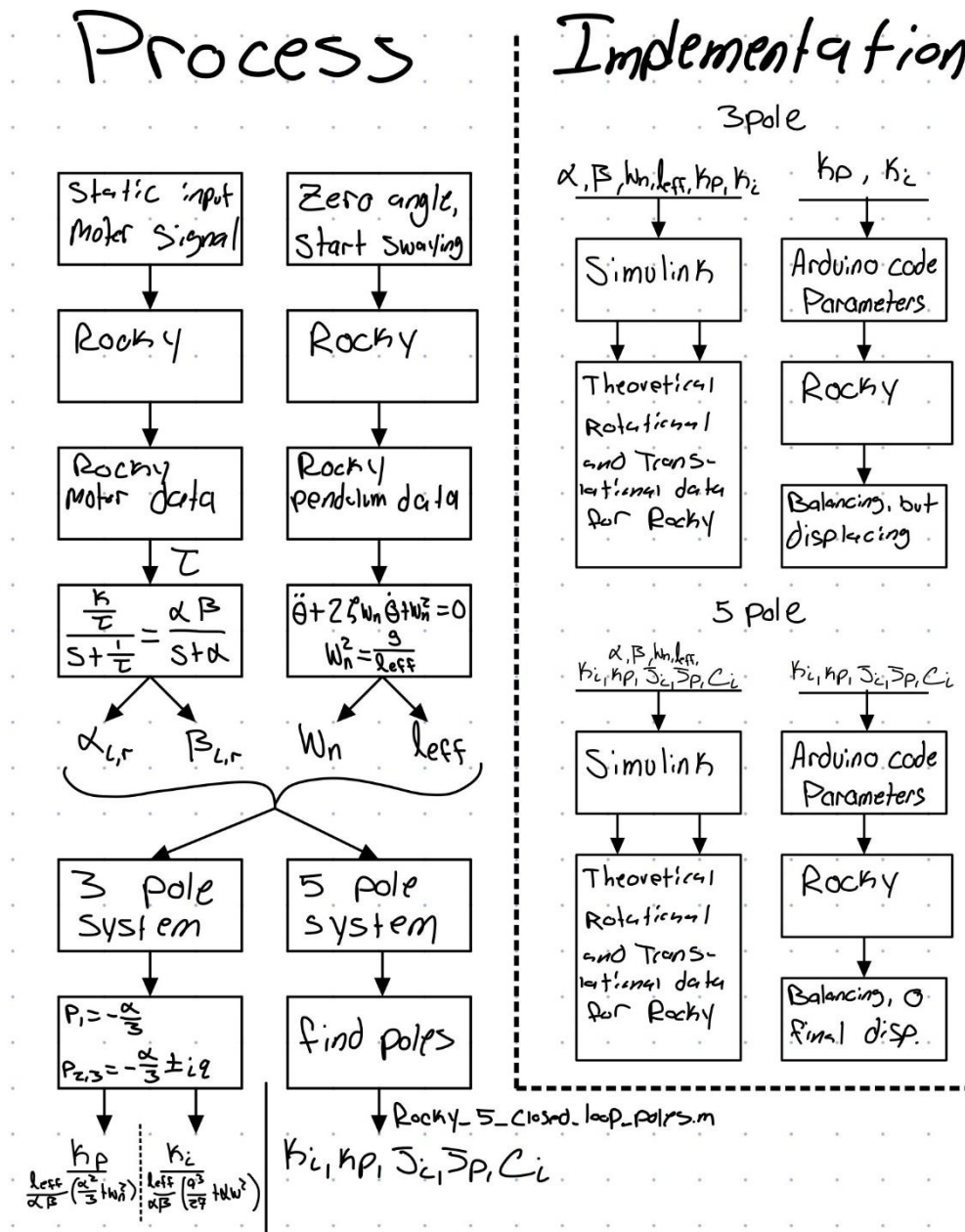


Figure 11: Velocity vs Time from 5 Pole Simulink Simulation

The difference between the theta values was as such: in the three pole model, theta goes to zero, then remains there as velocity and position keep increasing. In the five pole model, theta oscillates to zero before being disturbed again. For position, in the three pole model

the x value increases exponentially as the rocky stays upright. In the five-pole model, the x value returns to zero each time. Lastly, the velocity in the three-pole model rises every disturbance, because the rocky speeds up to avoid falling. Because the rocky oscillates to stop, the velocity oscillates before resetting to zero each time in the five-pole model.

Below is our block diagram of how we did the rocky project.



Arduino Code to control the rocky:

```
// Start the robot flat on the ground
// compile and load the code
// wait for code to load (look for "done uploading" in the Arduino IDE)
// wait for red LED to flash on board
// gently lift body of rocky to upright position
// this will enable the balancing algorithm
// wait for the buzzer
// let go
//
// The balancing algorithm is implemented in BalanceRocky()
// which you should modify to get the balancing to work
//
```

```
#include <Balboa32U4.h>
#include <Wire.h>
#include <LSM6.h>
#include "Balance.h"
```

```
extern int32_t angle_accum;
extern int32_t speedLeft;
extern int32_t driveLeft;
extern int32_t distanceRight;
extern int32_t speedRight;
extern int32_t distanceLeft;
extern int32_t distanceRight;
float speedCont = 0;
float displacement_m = 0;
int16_t limitCount = 0;
uint32_t cur_time = 0;
float distLeft_m;
float distRight_m;
```

```
extern uint32_t delta_ms;
float measured_speedL = 0;
float measured_speedR = 0;
float desSpeedL=0;
float desSpeedR =0;
float dist_accumL_m = 0;
```

```

float dist_accumR_m = 0;
float dist_accum = 0;
float speed_err_left = 0;
float speed_err_right = 0;
float speed_err_left_acc = 0;
float speed_err_right_acc = 0;
float errAccumRight_m = 0;
float errAccumLeft_m = 0;
float prevDistLeft_m = 0;
float prevDistRight_m = 0;
float angle_rad_diff = 0;
float angle_rad;           // this is the angle in radians
float angle_rad_accum = 0; // this is the accumulated angle in radians
float angle_prev_rad = 0; // previous angle measurement
extern int32_t displacement;
int32_t prev_displacement=0;
uint32_t prev_time;

#define G_RATIO (162.5)

LSM6 imu;
Balboa32U4Motors motors;
Balboa32U4Encoders encoders;
Balboa32U4Buzzer buzzer;
Balboa32U4ButtonA buttonA;

#define FIXED_ANGLE_CORRECTION (0.278) // ***** Replace the value 0.25 with the
value you obtained from the Gyro calibration procedure

////////////////////////////////////
////////////////////////////////////
// This is the main function that performs the balancing
// It gets called approximately once every 10 ms by the code in loop()
// You should make modifications to this function to perform your
// balancing

```



```

    // right to left. This helps ensure that the Left and Right motors are balanced

    // *** enter equations for input signals for v_c (left and right) in terms of
    the variables available ****
    v_c_R = v_d - (Jp * (measured_speedR) + Ji * (distLeft_m) + Ci *
(dist_accum));
    v_c_L = v_d - (Jp * (measured_speedL) + Ji * (distRight_m) + Ci *
(dist_accum));

    // save desired speed for debugging
    desSpeedL = v_c_L;
    desSpeedR = v_c_R;

    // the motor control signal has to be between +- 300. So clip the values to
    be within that range
    // here
    if(v_c_L > 300) v_c_L = 300;
    if(v_c_R > 300) v_c_R = 300;
    if(v_c_L < -300) v_c_L = -300;
    if(v_c_R < -300) v_c_R = -300;

    // Set the motor speeds
    motors.setSpeeds((int16_t) (v_c_L), (int16_t)(v_c_R));

}

```

```

void setup()
{
    // Uncomment these lines if your motors are reversed.
    // motors.flipLeftMotor(true);
    // motors.flipRightMotor(true);

    Serial.begin(9600);
    prev_time = 0;
}

```



```

displacement = 0;
ledYellow(0);
ledRed(1);
balanceSetup();
ledRed(0);
angle_accum = 0;

ledGreen(0);
ledYellow(0);
}

```

```

int16_t time_count = 0;
extern int16_t angle_prev;
int16_t start_flag = 0;
int16_t start_counter = 0;
void lyingDown();
extern bool isBalancingStatus;
extern bool balanceUpdateDelayedStatus;

void UpdateSensors()
{
    static uint16_t lastMillis;
    uint16_t ms = millis();

    // Perform the balance updates at 100 Hz.
    balanceUpdateDelayedStatus = ms - lastMillis > UPDATE_TIME_MS + 1;
    lastMillis = ms;

    // call functions to integrate encoders and gyros
    balanceUpdateSensors();

    if (imu.a.x < 0)
    {
        lyingDown();
        isBalancingStatus = false;
    }
    else
    {
        isBalancingStatus = true;
    }
}

```

```

void GetMotorAndAngleMeasurements()
{
    // convert distance calculation into meters
    // and integrate distance
    distLeft_m = ((float)distanceLeft)/((float)G_RATIO)/12.0*80.0/1000.0*3.14159;
    distRight_m =
((float)distanceRight)/((float)G_RATIO)/12.0*80.0/1000.0*3.14159;
    dist_accum += (distLeft_m+distRight_m)*0.01/2.0;

    // compute left and right wheel speed in meters/s
    measured_speedL = speedLeft/((float)G_RATIO)/12.0*80.0/1000.0*3.14159*100.0;
    measured_speedR = speedRight/((float)G_RATIO)/12.0*80.0/1000.0*3.14159*100.0;

    prevDistLeft_m = distLeft_m;
    prevDistRight_m = distRight_m;

    // this integrates the angle
    angle_rad_accum += angle_rad*0.01;
    // this is the derivative of the angle
    angle_rad_diff = (angle_rad-angle_prev_rad)/0.01;
    angle_prev_rad = angle_rad;

}

void balanceResetAccumulators()
{
    errAccumLeft_m = 0.0;
    errAccumRight_m = 0.0;
    speed_err_left_acc = 0.0;
    speed_err_right_acc = 0.0;
}

void loop()
{
    static uint32_t prev_print_time = 0;    // this variable is to control how often
    we print on the serial monitor

```

```

    int16_t distanceDiff;    // this stores the difference in distance in encoder
                             clicks that was traversed by the right vs the left wheel
    static float del_theta = 0;
    char enableLongTermGyroCorrection = 1;

    cur_time = millis();          // get the current time in milliseconds

    if((cur_time - prev_time) > UPDATE_TIME_MS){
        UpdateSensors();          // run the sensor updates.

        // calculate the angle in radians. The FIXED_ANGLE_CORRECTION term comes from
        the angle calibration procedure (separate sketch available for this)
        // del_theta corrects for long-term drift
        angle_rad = ((float)angle)/1000/180*3.14159 - FIXED_ANGLE_CORRECTION -
        del_theta;

        if(angle_rad > 0.1 || angle_rad < -0.1)    // If angle is not within +- 6
        degrees, reset counter that waits for start
        {
            start_counter = 0;
        }

        if(angle_rad > -0.1 && angle_rad < 0.1 && ! start_flag)
        {
            // increment the start counter
            start_counter++;
            // If the start counter is greater than 30, this means that the angle has
            been within +- 6 degrees for 0.3 seconds, then set the start_flag
            if(start_counter > 30)
            {
                balanceResetEncoders();
                start_flag = 1;
                buzzer.playFrequency(DIV_BY_10 | 445, 1000, 15);
                Serial.println("Starting");
                ledYellow(1);
            }
        }
    }

    // every UPDATE_TIME_MS, if the start_flag has been set, do the balancing

```

```

if(start_flag)
{
    GetMotorAndAngleMeasurements();
    if(enableLongTermGyroCorrection)
        del_theta = 0.999*del_theta + 0.001*angle_rad; // assume that the robot is
standing. Smooth out the angle to correct for long-term gyro drift

    // Control the robot
    BalanceRocky();
}
prev_time = cur_time;
}
// if the robot is more than 45 degrees, shut down the motor
if(start_flag && angle_rad > .78)
{
    motors.setSpeeds(0,0);
    start_flag = 0;
}
else if(start_flag && angle_rad < -0.78)
{
    motors.setSpeeds(0,0);
    start_flag = 0;
}

// kill switch
if(buttonA.getSingleDebouncePress())
{
    motors.setSpeeds(0,0);
    while(!buttonA.getSingleDebouncePress());
}

if(cur_time - prev_print_time > 103) // do the printing every 105 ms. Don't
want to do it for an integer multiple of 10ms to not hog the processor
{
    Serial.print(angle_rad);
    Serial.print("\t");
    Serial.print(distLeft_m);
    Serial.print("\t");
    Serial.print(measured_speedL);
    Serial.print("\t");
    Serial.print(measured_speedR);
    Serial.print("\t");
    Serial.println(speedCont);
}

```

```
        prev_print_time = cur_time;
    }

}
```